

TypeScript - I

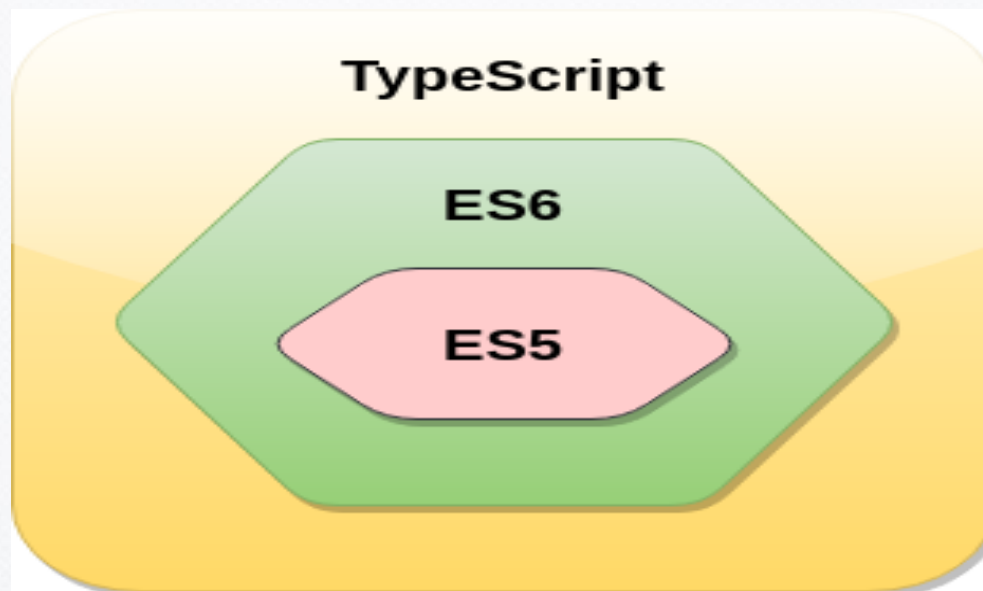


Session 1



- Giới thiệu về TypeScript
- Kiểu dữ liệu và biến
- Toán tử
- Cấu trúc điều kiện
- Vòng lặp
- Hàm

- TypeScript (TS) là một ngôn ngữ lập trình hướng đối tượng mã nguồn mở, và là một phiên bản cải tiến nâng cấp của JavaScript (JS) bởi việc bổ sung tùy chọn kiểu tĩnh và lớp hướng đối tượng mà điều này không có ở Javascript , sẽ được biên dịch thành JavaScript thuần. TS chứa mọi thành phần của JS.
- TS là ngôn ngữ được thiết kế cho quá trình phát triển các ứng dụng JS có quy mô lớn, có thể sử dụng để phát triển các ứng dụng chạy ở client-side (Angular2) và server-side (NodeJS). TS chính là phiên bản ES6 của JS với một số đặc điểm bổ sung.



- TypeScript không thể thực thi trực tiếp trên trình duyệt. TS cần một trình biên dịch để biên dịch file và sinh ra file JS để thực thi trên trình duyệt. File mã nguồn TypeScript sẽ có phần mở rộng là ".ts". Ta có thể sử dụng bất kỳ file ".js" nào bằng cách đổi tên nó thành file có đuôi ".ts".
- TypeScript sử dụng trình biên dịch TSC (TypeScript Compiler) để chuyển đổi mã TypeScript (.ts file) thành mã JavaScript (.js file).

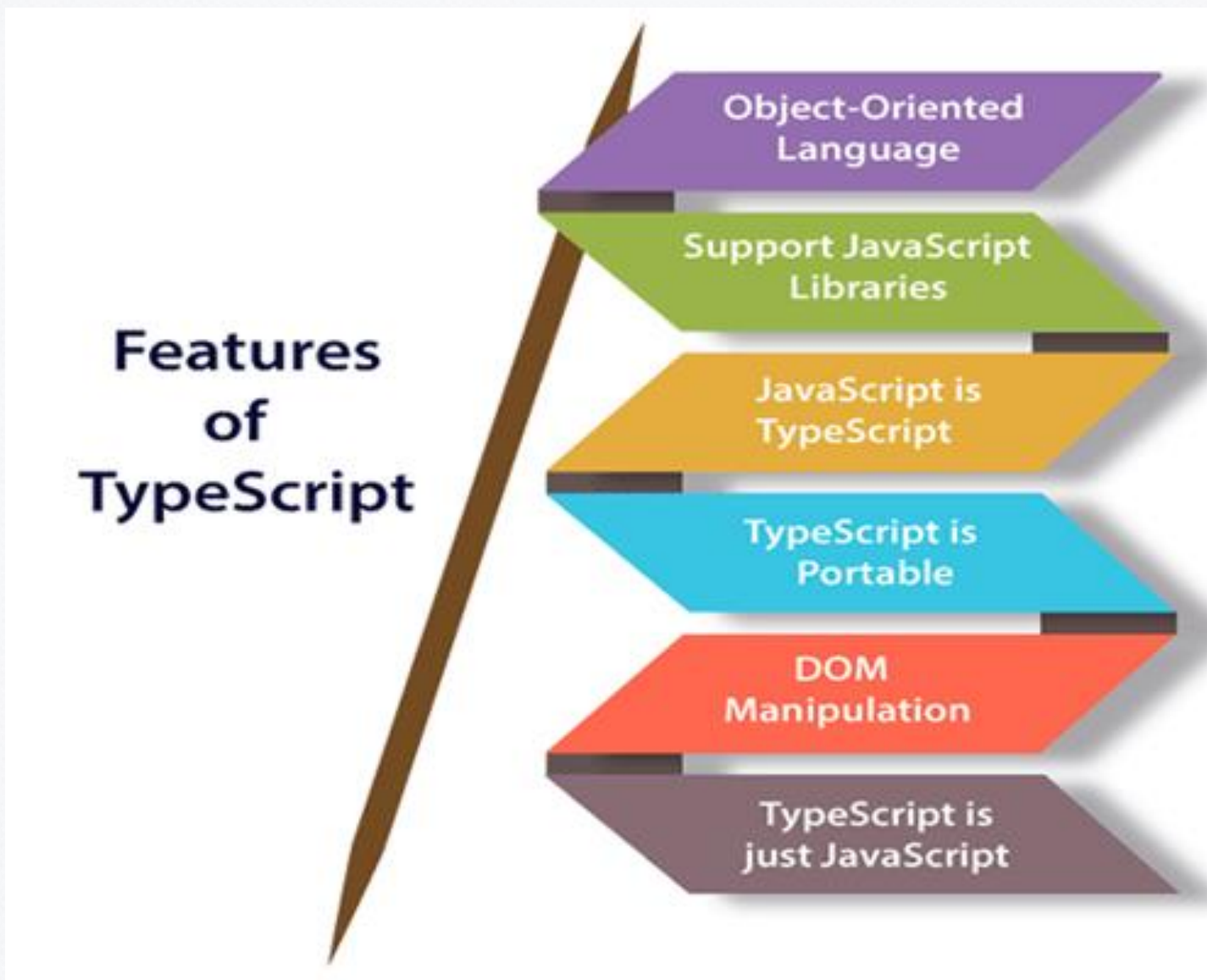


- Năm 2010: TypeScript được phát triển bởi Anders Hejlsberg là một thành viên của nhóm phát triển ngôn ngữ C# tại Microsoft
- Năm 2012: Phiên bản đầu tiên được phát hành (TypeScript 0.8)
- Hiện tại TS được duy trì bởi Microsoft dưới bản quyền Apache 2
- Phiên bản mới nhất hiện tại là TypeScript 5.x

Tại sao nên sử dụng TypeScript?

- TypeScript được sử dụng vì một số lợi ích như sau.
 - TypeScript hỗ trợ các tính năng như Static typing, Strongly type, Modules, Optional Parameters v.v...
 - TypeScript hỗ trợ các đặc điểm của OOP như: classes, interfaces, inheritance, generics v.v...
 - TypeScript dễ học, đơn giản, cho phép học nhanh chóng.
 - TypeScript cung cấp cơ chế cho phép phát hiện lỗi vào thời điểm biên dịch. Các lỗi nếu có sẽ được làm rõ trước khi đoạn mã được thực thi.
 - TypeScript hỗ trợ tất cả các thư viện JavaScript vì TS chính là superset của JavaScript.
 - TypeScript hỗ trợ khả năng tái sử dụng thông qua cơ chế inheritance trong OOP.
 - TypeScript làm cho việc phát triển app trở nên nhanh chóng và dễ dàng nhất có thể, cùng với các công cụ hỗ trợ của TypeScript cung cấp các tính năng như autocompletion, type checking, và source documentation.
 - TypeScript có một definition file với phần mở rộng .d.ts extension để cung cấp định nghĩa cho các thư viện JavaScript mở rộng.
 - TypeScript hỗ trợ đầy đủ các tính năng JavaScript mới nhất, bao gồm cả ECMAScript 2015.
 - TypeScript cung cấp mọi lợi ích của ES6 cùng với nhiều hiệu suất hơn, do đó các developers có thể tiết kiệm nhiều thời gian với TypeScript.

Các đặc điểm của TypeScript

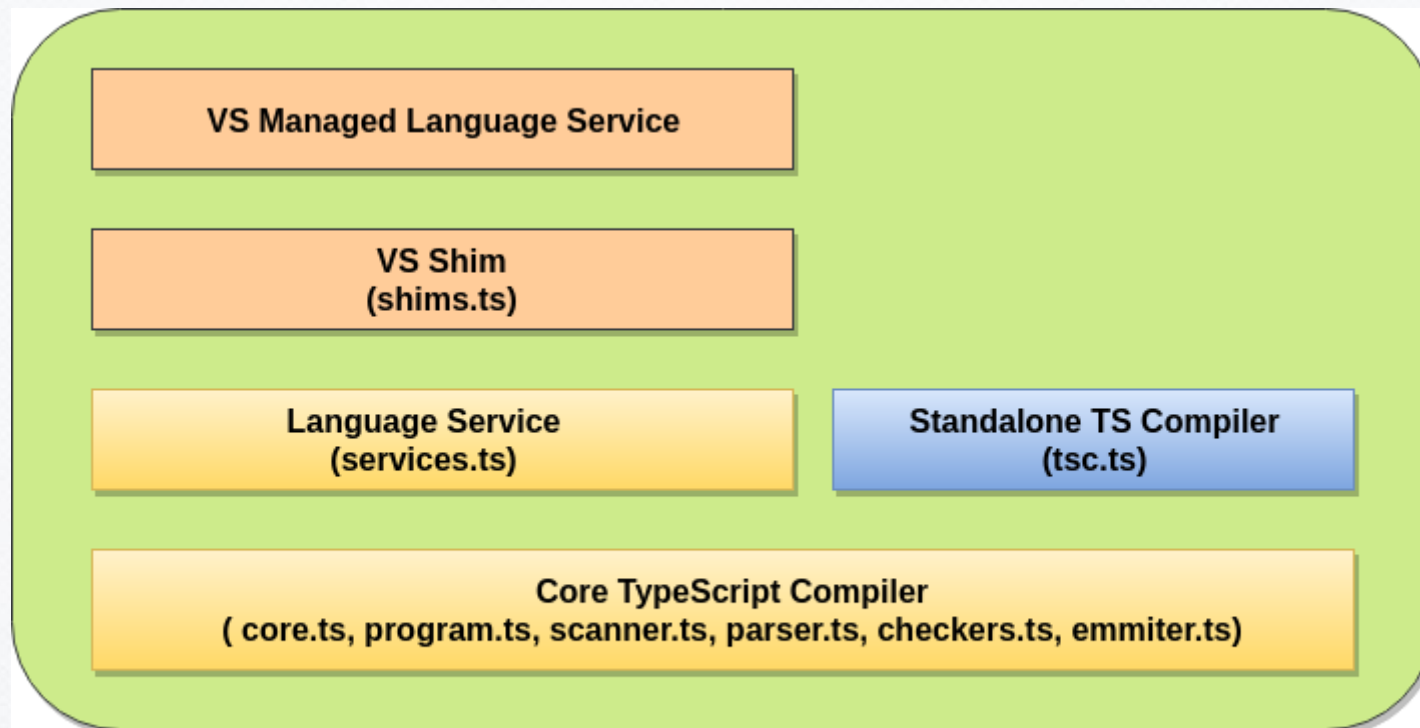


Các đặc điểm của TypeScript

- **Object-Oriented language:** TypeScript hỗ trợ đầy đủ các đặc điểm của OOP như: classes, interfaces, inheritance, modules v.v... Trong TypeScript, ta có thể viết mã cho cả client-side cũng như server-side.
- **TypeScript hỗ trợ các thư viện JavaScript:** TypeScript hỗ trợ các phần tử JavaScript, nó cho phép các developers có thể sử dụng mã JavaScript với TypeScript. Vì vậy, ta có thể sử dụng dễ dàng các JavaScript frameworks, tools, và các thư viện khác.
- **JavaScript là TypeScript:** Mã được viết bằng JavaScript với phần mở rộng .js có thể được chuyển đổi thành TypeScript bằng cách đổi phần mở rộng từ .js thành .ts và được biên dịch với các file TypeScript khác.
- **TypeScript là portable:** TypeScript là portable vì có thể được thực thi trong bất kỳ trình duyệt, thiết bị hay các hệ điều hành nào. TS có thể được thực thi trên bất kỳ môi trường nào mà JavaScript có thể thực thi.
- **DOM Manipulation:** TypeScript có thể được dùng để thao tác với DOM, như việc thêm/xóa các phần tử tương tự như JavaScript.
- **TypeScript chỉ là JS:** TypeScript code không được thực thi trực tiếp trên bất kỳ trình duyệt nào. Code được viết bằng TypeScript sẽ được biên dịch và chuyển đổi thành JavaScript để thực thi. Tiến trình này được gọi là **Trans-plied**. Với sự hỗ trợ của mã JavaScript, trình duyệt có thể đọc mã TypeScript và hiển thị kết quả.

Các thành phần của TypeScript

- Ngôn ngữ TypeScript được chia thành 3 layers chính. Mỗi layer sẽ được chia thành các sublayers hoặc các components. Ba layer bao gồm:
 1. Language
 2. The TypeScript Compiler
 3. The TypeScript Language Services



Components of TypeScript

- 1. Language

- Gồm các thành phần của ngôn ngữ TypeScript như cú pháp, các từ khóa, toán tử và các type annotations.

- 2. The TypeScript Compiler

- Trình biên dịch TypeScript (TSC) sẽ chuyển chương trình TypeScript thành mã JavaScript tương đương. TSC cũng sẽ thực hiện quá trình parsing, type checking mã nguồn TypeScript trong lúc chuyển đổi.



- Ví dụ

```
$ tsc helloworld.ts // It compiles the TS file helloworld into the helloworld.js file.
```

- 3. TypeScript Language Services

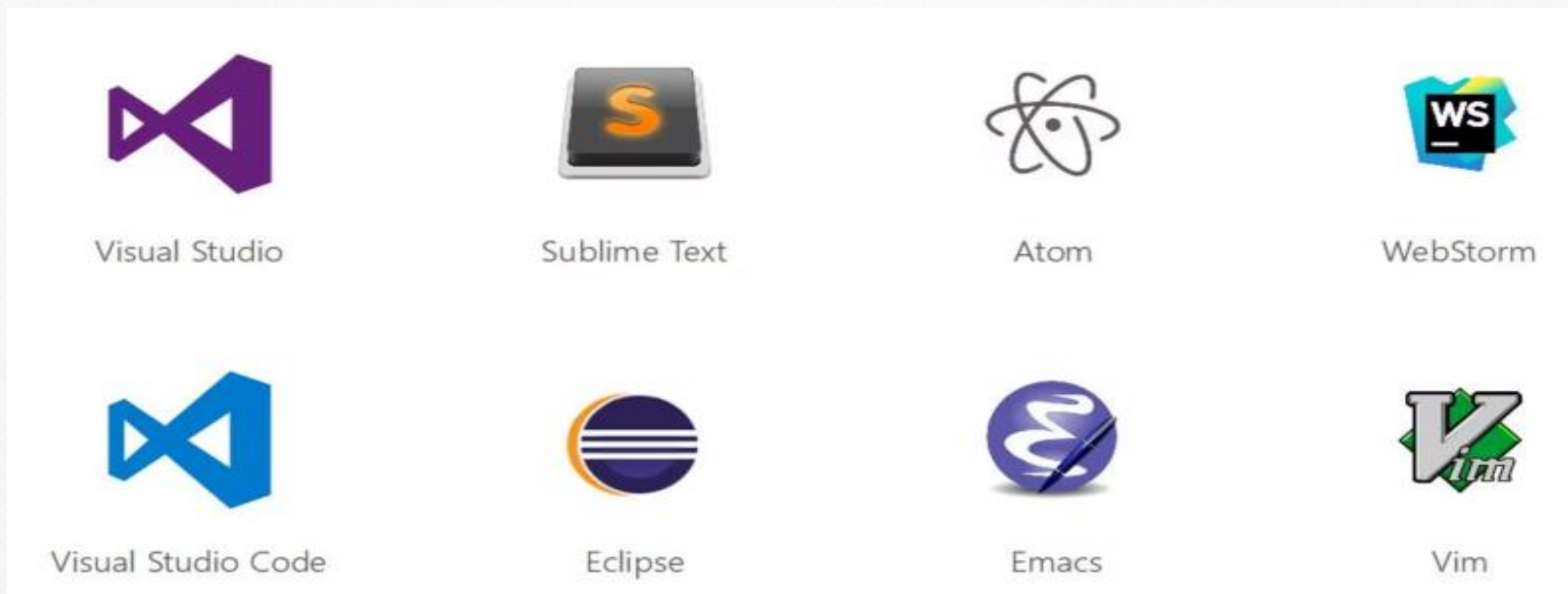
- Language service cung cấp các thông tin hỗ trợ các trình soạn thảo và các công cụ khác để có các đặc điểm hỗ trợ tốt hơn, chẳng hạn như automated refactoring và IntelliSense. Nó cũng hỗ trợ các thao tác soạn thảo thông dụng như code formatting và outlining, colorization, statement completion, signature help v.v...

1. Download và cài đặt Node.js (npm hoặc yarn)
(<https://nodejs.org>)

2. Cài đặt TypeScript compiler
npm install -g typescript

3. Text Editor hoặc IDE (Visual Studio Code, Subline Text, Visual studio, Eclipse, WebStorm, Vim.....).

Ghi chú: Luvia quy định sử dụng IDE Visual Studio Code.



Chương trình TypeScript đầu tiên



- Bước 1: Tạo một thư mục chứa source code
- Bước 2: Dùng Visual Studio Code mở thư mục đã tạo
- Bước 3: Tạo 1 file .ts trong thư mục. Ví dụ: typescript_ex1.ts
- Bước 4: Soạn thảo code :

```
typescript_ex1.ts

console.log("Hello World!");

function sayHello(name: String) {
  console.log(`Hello ${name}`);
}

sayHello("Tom");
```

- Bước 5: Biên dịch chương trình : `tsc typescript_ex1.ts`
- Bước 6: Chạy chương trình: `node typescript_ex1.js`

Biến và kiểu dữ liệu



- Biến là gì?
 - Biến là một vùng nhớ dùng để lưu trữ giá trị mà giá trị đó có thể thay đổi khi chương trình thực thi.
 - Mỗi biến gắn liền với một kiểu dữ liệu và một định danh duy nhất gọi là tên biến
- Khai báo biến sử dụng
 - Sử dụng từ khóa var, let, const để khai báo

Cú pháp:

Cách 1: `var [Tên biến]:[kiểu_dữ_liệu] = value;`

Cách 2: `var [Tên biến]:[kiểu_dữ_liệu];`

Cách 3: `var [Tên biến] = value;`

Cách 4: `var [Tên biến];`

Ví dụ:

```
var name: string = "Jone";
```

```
var score: number = 8.5;
```

- Từ khóa **const** được sử dụng để khai báo một hằng số (constant), biến này phải được gán giá trị ngay tại thời điểm khai báo. Sau đó, bạn không thể gán giá trị mới cho biến này

Cú pháp:

Cách 1: `const [Tên biến]:[kiểu_dữ_liệu] = value;`

Cách 2: `const [Tên biến] = value;`

Ví dụ:

```
var name: string = "Jone";
```

```
var score: number = 8.5;
```


- Khai báo biến có thể sử dụng từ khóa “let” hoặc “var”
- Từ khóa let được bắt đầu sử dụng từ ECM6, tương tự như “var” nhưng có một vài điểm khác biệt:
 - Khai báo biến với **var** được xác định có phạm vi toàn cục (global) hoặc phạm vi cục bộ hàm nếu khai báo trong hàm
 - Khai báo biến với **let** được xác định có phạm vi trong một khối. Một khối là một đoạn mã được bao bởi cặp dấu mở ngoặc nhọn và đóng ngoặc nhọn { }. Mọi lệnh trong cặp dấu là một khối mã (Block).

Ví dụ:

```
// Note the scope differences
function scopeTesting()
{
    var testVar = 100;

    for (let testLet = 0; testLet <= 10; testLet++)
    {
        console.log ("The value of Let " + testLet);
        console.log ("The value of Var " + testVar);
    }

    console.log ("The value of Var" + testLet);
    console.log ("The value of Var" + testVar);
}
```

- Kiểu dữ liệu cơ bản

- TypeScript có 5 built-in data types như sau.

- ◆ Number: kiểu số

- Ví dụ: `var height: number = 8;`

- ◆ String: Kiểu chuỗi kí tự

- Ví dụ: `var name: string = "David";`

- ◆ Boolean: Kiểu logic trả về kết quả true hoặc false

- Ví dụ: `var isDone: boolean = true`

- ◆ Any: kiểu bất kì

- Ví dụ: `var notSure: any = 4;`

- `notSure = "maybe a string instead"; // hoàn toàn được`

- ◆ Void: Cũng giống như any nhưng void được sử dụng là đầu ra của hàm

- Ví dụ:

```
function warnUser(): void {  
    alert("This is my warning message");  
}
```

Kiểu dữ liệu

- Kiểu do người dùng định nghĩa (User-Defined DataType)
 - Array
 - Tuple
 - Enum
 - Union
 - Dictionary
 - Object

- **Mảng:** Có 2 kiểu khai báo tương đương với nhau trong TypeScript. Ví dụ:

Example: Array Declaration and Initialization

```
let fruits: Array<string>;  
fruits = ['Apple', 'Orange', 'Banana'];  
  
let ids: Array<number>;  
ids = [23, 34, 100, 124, 44];
```

- **Tuple:** Kiểu dữ liệu tuple là 1 mảng cố định số phần tử mà mỗi phần tử có kiểu dữ liệu khác nhau. Ví dụ

Example: Tuple vs Other Data Types

```
var empId: number = 1;  
var empName: string = "Steve";  
  
// Tuple type variable  
var employee: [number, string] = [1, "Steve"];
```


- **ENUM:** TypeScript hỗ trợ cả string-based và numeric-based enums. Mặc định, enums bắt đầu đánh số các phần tử bắt đầu từ số 0 (ta có thể thay đổi thứ tự này hoặc gán giá trị cho các phần tử trong enums).

Example: Enum as Return Type

```
enum PrintMedia {  
    Newspaper = 1,  
    Newsletter,  
    Magazine,  
    Book  
}  
  
function getMedia(mediaName: string): PrintMedia {  
    if ( mediaName === 'Forbes' || mediaName === 'Outlook') {  
        return PrintMedia.Magazine;  
    }  
}  
  
let mediaType: PrintMedia = getMedia('Forbes'); // returns Magazine
```

- **UNION:** Trong TypeScript, một biến hoặc một tham số cho hàm (parameter for function) có thể có nhiều kiểu kết hợp nhau, chúng gọi là Union type, ví dụ:

Example: Union

```
let code: (string | number);
code = 123; // OK
code = "ABC"; // OK
code = false; // Compiler Error

let empId: string | number;
empId = 111; // OK
empId = "E111"; // OK
empId = true; // Compiler Error
```

Example: Function Parameter as Union Type

```
function displayType(code: (string | number))
{
    if(typeof(code) === "number")
        console.log('Code is number.')
    else if(typeof(code) === "string")
        console.log('Code is string.')
}

displayType(123); // Output: Code is number.
displayType("ABC"); // Output: Code is string.
displayType(true); //Compiler Error: Argument of type 'true' is not assignable to a parameter of type string | number
```

• DICTIONARY:

- TypeScript không có kiểu với tên là dictionary, tuy nhiên trong TS có kiểu Map cho phép lưu trữ một tuyển tập các cặp key-value. Ta có thể khởi tạo dictionary bằng cách sử dụng map thông qua từ khóa new và Map.
- Các kiểu hợp lệ của key-value là string, number, boolean, class, interface.

```
let employees = new Map<string, string>();
employees.set("name", "john");

// Checking for the key exist :
employees.has("name"); // true

// get a value by a key:
employees.get("name"); // john

// delete an item by a key:
employees.delete("name");
// iterate key and values
employees.forEach((item, key) => console.log(item));

// remove all from map:
employees.clear();

// size: Number of elements in Map :
console.log(employees.size);

// get All keys with insertion order
let keys = Array.from(employees.keys());

// Extract values with insertion order
let values = Array.from(employees.values());
```

- **Object:** Kiểu object còn gọi là kiểu đối tượng, khai báo và sử dụng như các object javascript. Biến kiểu đối tượng được đặt trong dấu {}, các thành viên cách nhau bởi dấu phẩy. Từng thành là từng cặp name:value.
- Ví dụ:

```
const nhanvien = {  
    hoten:"Trần Đức Thịnh",  
    tuoi: 35,  
    phones: ['0917438274','095632421']  
}  
let ht = nhanvien.hoten;  
console.log(ht);//Trần Đức Thịnh
```


- Có 2 cách ép kiểu trong Typescript, cách 1 là dùng **<type>** còn cách 2 là dùng từ khóa **as**. Ví dụ:

Cách 1

```
let userInput:unknown = prompt("Mời nhập cái gì đó");  
let sokyту: number = ( <string> userInput ).length;  
console.log(sokyту);
```

Cách 2

```
let userInput:unknown = prompt("Mời nhập cái gì đó");  
let sokyту: number = (userInput as string).length;  
console.log(sokyту);
```

- Generic

- Là đặc điểm cho phép tạo ra một component có thể làm việc với nhiều kiểu dữ liệu khác nhau thay cho một kiểu duy nhất. Generic cho phép tạo ra các reusable components, để đảm bảo rằng chương trình có thể mở rộng hoặc có tính linh hoạt cao.
- TypeScript sử dụng generics với khai báo `<T>` để thể hiện types. Type của các generic functions cũng giống như các non-generic functions, với khai báo type parameters, tương tự như định nghĩa function thông thường.

```
function identity<T>(arg: T): T {  
    return arg;  
}  
  
let output1 = identity<string>("myString");  
let output2 = identity<number>( 100 );
```

- Decorators

➤ Là một kiểu dữ liệu đặc biệt được gắn với các thành phần cú pháp, VD như class declaration, method, property, accessor, và parameter. Decorator cung cấp cả annotations và cú pháp meta-programing cho các classes và functions để cung cấp thông tin mô tả cho các thành phần của class. Decorator được khai báo với ký hiệu "@".

➤ Ví dụ:

```
function f() {  
    console.log("f(): evaluated");  
    return function (target, propertyKey: string, descriptor: PropertyDescriptor) {  
        console.log("f(): called");  
    }  
}  
  
class C {  
    @f()  
    method() {}  
}
```

- Toán tử (Operator) là một phép tính dùng để thao tác trên các giá trị hoặc dữ liệu. Toán tử sẽ thao tác trên các toán hạng. Tất cả các toán tử trong JS đều có thể được sử dụng trong chương trình TS.
- Trong TypeScript, có những loại toán tử sau:
 - Toán tử đại số: +, -, *, /, %
 - Toán tử quan hệ (so sánh): ==, ===, !=, !==, >, >=, <, <=
 - Toán tử logical: &&, ||, !
 - Toán tử gán: =, +=, -=, *=, /=
 - Toán tử ternary (điều kiện): (condition) ? value1: value2;

- Toán tử type

- Typeof operator: Trả về kiểu dữ liệu của toán hạng.

- ◆ Ví dụ:

```
var num = 12  
console.log(typeof num);    //output: number
```

- Instanceof: Kiểm tra xem một đối tượng có thuộc một kiểu nào đó hay không.

Cấu trúc lệnh điều khiển



- Cấu trúc lệnh IF

- if statement

- Là dạng đơn giản nhất của decision making. Nó sẽ kiểm tra điều kiện, nếu điều kiện thỏa mãn thì sẽ thực thi thân của khối if.

```
if(condition) {  
    // code to be executed  
}
```

- if-else statement

- Cấu trúc if-else nếu biểu thức điều kiện trả về đúng và sai. Nếu điều kiện đúng, khối if sẽ được thực thi, nếu điều kiện sai thì khối else sẽ được thực thi.

```
if(condition) {  
    // code to be executed  
} else {  
    // code to be executed  
}
```

- if-else-if ladder

- Là cú pháp mở rộng của cấu trúc if-else. Cú pháp này cho phép đánh giá nhiều điều kiện trong chương trình. Cú pháp này còn được gọi là cấu trúc if else bậc thang.

Cú pháp:

```
if(condition1){
  //code to be executed if condition1 is true
}else if(condition2){
  //code to be executed if condition2 is true
}
else if(condition3){
  //code to be executed if condition3 is true
}
else{
  //code to be executed if all the conditions are false
}
```

• Ví dụ:

```
let marks = 95;
if(marks<50){
  console.log("fail");
}
else if(marks>=50 && marks<60){
  console.log("D grade");
}
else if(marks>=60 && marks<70){
  console.log("C grade");
}
else if(marks>=70 && marks<80){
  console.log("B grade");
}
else if(marks>=80 && marks<90){
  console.log("A grade");
}else if(marks>=90 && marks<100){
  console.log("A+ grade");
}else{
  console.log("Invalid!");
}
```


- Nested if statement

➤ Khối if bên trong sẽ phụ thuộc vào khối if bên ngoài.

```
if(condition1) {  
    //Nested if else inside the body of "if"  
    if(condition2) {  
        //Code inside the body of nested "if"  
    }  
    else {  
        //Code inside the body of nested "else"  
    }  
}  
else {  
    //Code inside the body of "else."  
}
```

- If Shorthand

➤ Câu điều kiện if () ... else ... luôn được sử dụng rất nhiều trong khi code, ko chỉ Javascript mà còn ở tất cả các ngôn ngữ khác. Việc sử dụng toán tử 3 ngôi sẽ giúp chúng ta rút ngắn được rất nhiều code.

➤ Ví dụ:

```
const x = 100;  
let result;  
if (x < 1000) {  
    result = "nhỏ hơn 1000";  
} else {  
    result = "lớn hơn hoặc bằng 1000";  
}
```

Rút gọn thành:

```
const x = 100;  
const result = (x < 1000) ? "nhỏ hơn 1000" : "lớn hơn hoặc bằng 1000";
```

- **Cấu trúc lệnh Switch..Case**

- Lệnh switch trong TypeScript sẽ thực thi một câu lệnh dựa trên nhiều điều kiện, nhiều nhánh của chương trình. Lệnh này sẽ kiểm tra một biểu thức dựa trên giá trị của nó thuộc một trong những kiểu sau: Boolean, number, byte, short, int, long, enum type, string v.v... Lệnh switch sẽ gồm một khối lệnh tương ứng với từng giá trị. Khi giá trị của một nhánh case khớp với giá trị của biểu thức, khối lệnh tương ứng sẽ được thực thi. Lệnh switch sẽ tương đương với khối if-else-if ladder.

- **Cú pháp:**

```
switch(expression){  
  
    case expression1:  
        //code to be executed;  
        break; //optional  
  
    case expression2:  
        //code to be executed;  
        break; //optional  
    .....  
  
    default:  
        //when no case is matched, this block will be executed;  
        break; //optional  
}
```

- Ví dụ switch:

```
let day : number = 4;

switch (day) {
  case 0:
    console.log("It is a Sunday.");
    break;
  case 1:
    console.log("It is a Monday.");
    break;
  case 2:
    console.log("It is a Tuesday.");
    break;
  case 3:
    console.log("It is a Wednesday.");
    break;
  case 4:
    console.log("It is a Thursday.");
    break;
  case 5:
    console.log("It is a Friday.");
    break;
  case 6:
    console.log("It is a Saturday.");
    break;
  default:
    console.log("No such day exists!");
    break;
}
```


- Vòng lặp For

- Là vòng lặp biết trước số lần lặp. Typescript hỗ trợ cho chúng ta 3 loại vòng lặp:

1. for loop
2. for..of loop
3. for..in loop

- Vòng lặp For Loop

- Cú pháp:

```
for(<khởi tạo>; <điều kiện lặp>; <bước nhảy>) {  
    <khối lệnh>;  
}
```

- Ví dụ:

```
for (let i = 0; i < 3; i++) {  
    console.log ("Block statement execution no." + i);  
}
```

- TypeScript for..of loop

- Vòng lặp for..of được dùng để duyệt và truy cập đến các phần tử trong một mảng, string, set, map, list, hoặc tuple.
- Cú pháp:

```
let arr = [1, 2, 3, 4, 5];  
  
for (var val of arr) {  
    console.log(val);  
}
```

- TypeScript for..in loop

- Vòng lặp for..in được sử dụng với mảng, list, hoặc tuple. Vòng lặp này sẽ cho phép duyệt qua một danh sách hoặc một tuyển tập và trả về chỉ số qua mỗi lần lặp.

```
for (var val in list) {  
    //statements  
}
```

- Vòng lặp While

- Vòng lặp while được thực thi chừng nào điều kiện còn đúng, và là loại vòng lặp mà ta chưa biết trước số lần lặp. Cú pháp như sau:

```
while (condition)
{
    //code to be executed
}
```

- Ví dụ

```
let i: number = 2;
while (i < 4) {
    console.log( "Block statement execution no." + i )
    i++;
}
```

- Vòng lặp do-while

- Vòng lặp do-while cũng thực thi chừng nào điều kiện còn đúng, và được sử dụng khi ta chưa biết trước số lần lặp. Tuy nhiên, vòng lặp do-while sẽ thực hiện vòng lặp trước rồi mới kiểm tra điều kiện, vì vậy nếu điều kiện sai thì vòng lặp do-while sẽ được thực hiện ít nhất một lần.

```
do{  
    //code to be executed  
}while (condition);
```

- Ví dụ:

```
let i: number = 2;  
do {  
    console.log("Block statement execution no." + i )  
    i++;  
} while ( i < 4)
```


Hàm trong TypeScript



Hàm (Function)

- Function là một khối lệnh trong đó có nhiều dòng lệnh. Trong Typescript có 2 dạng là name function (tên) và anonymous function(không tên).

- Name Function

- Cú pháp:

```
function name(parameter: type, parameter:type,...): returnType {  
    // do something  
}
```

- Ví dụ:

```
function add(a: number, b: number): number {  
    return a + b;  
}  
add(2,3); // returns 5
```

- Anonymous Function

- Cú pháp:

- ```
let res = function([arguments]) { }
```

- Ví dụ:

- ```
// Anonymous function
```

- ```
let myAdd = function (x: number, y: number) : number {
 return x + y;
};
```

- ```
// Anonymous function call
```

- ```
console.log(myAdd(5,3)); // output 8
```

- Arrow Function

- Cú pháp:

- (parameter1, parameter2, ..., parameterN) => expression;

- Ví dụ:

- 1. arrow function with parameter

- let sum = (a: number, b: number): number => {  
    return a + b;  
}

- console.log(sum(20, 30)); //returns 50

- 2. arrow function without parameter

- let Print = () => console.log("Hello JavaTpoint!");  
Print();



- Default parameters: có thể gán giá trị mặc định cho tham số

➤ Cú pháp:

```
function function_name(param1[:type],param2[:type] =
 default_value) {

}
```

➤ Ví dụ:

```
function calculate_discount(price:number,rate:number = 0.50) {
 var discount = price * rate;
 console.log("Discount Amount: ",discount);
}
calculate_discount(1000) ;
calculate_discount(1000,0.30);
```

- Optional parameters: có thể truyền giá trị cho tham số hoặc không

➤ Cú pháp:

```
function function_name (param1[:type], param2[:type],
 param3[?:type])
```

➤ Ví dụ:

```
function disp_details(id:number,name:string,mail_id?:string) {
 console.log("ID:", id);
 console.log("Name",name);
 if(mail_id!==undefined)
 console.log("Email Id",mail_id);
}
disp_details(123,"John");
disp_details(111,"mary","mary@xyz.com");
```

- Rest parameters: cho phép truyền một hoặc nhiều tham số

➤ Cú pháp:

```
function function_name (...params [:type])
```

➤ Ví dụ:

```
function addNumbers(...nums:number[]) {
 var i;
 var sum:number = 0;
 for(i = 0;i<nums.length;i++) {
 sum = sum + nums[i];
 }
 console.log("sum of the numbers",sum)
}
addNumbers(1,2,3)
addNumbers(10,10,10,10,10)
```

## ➤ Bài 1

Hãy phát triển một chương trình TS cho phép thực hiện khai báo 2 biến, rồi tính toán các phép tính cộng, trừ, nhân, chia, sau đó in ra kết quả của chương trình.

## ➤ Bài 2

Viết một Chương trình cho phép tính tiền điện. Mỗi hộ gia đình sẽ được sử dụng 100 số điện đầu tiên (Giá rẻ ) với giá 450D/1KW. Công thức tính tiền điện cụ thể như sau:

Từ số 101 – 200: Giá tiền sẽ là 600/KW

Từ số 201 – 300: Giá tiền sẽ là 750/KW

Từ số 301 – 500: Giá tiền sẽ là 900/KW

Từ số 501 – 1000: Giá tiền sẽ là 1000/KW

Từ số 1000 trở lên: Giá tiền sẽ là 1200/KW

Yêu cầu: Khai báo KW tiêu thụ điện. Chương trình sẽ tính và in ra tổng số tiền mà ta phải nộp trước và sau khi tính thuế VAT (Thuế suất VAT = 10% tiền điện).



## ➤ Bài 3

Hãy viết chương trình TS cho phép tính và in ra tổng của 100 số nguyên đầu tiên (1-100), dùng vòng lặp While.

Viết chương trình TS có sử dụng vòng lặp While cho phép in ra những số chia hết cho 3 trong khoảng từ 20-50



- Giới thiệu về TypeScript
- Kiểu dữ liệu và biến
- Toán tử
- Cấu trúc điều kiện
- Vòng lặp